PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
46504-04 · Construção de Software · Turma 330 · 2026/2
Prof. Michael da Costa Mora

PROJETO INTEGRADOR · PACOTE 2

PulsePass

Arquitetura e Infraestrutura de Desenvolvimento

Arquitetura C4 · Modelo de Dados Conceitual · Mensageria Kafka · Pipeline de Desenvolvimento · Gerenciamento da Configuração

Integrantes: Gabriel Kowaleski e Mayra Bordin
Entrega original: 4 de novembro de 2026
Versão atualizada para o Pacote 3 (ver seção 8)

Sumário

1. **Visão geral e decisões arquiteturais** – premissas, stack, ADRs e rastreabilidade com os RNFs
2. **Arquitetura no Modelo C4** – Nível 1 (Contexto), Nível 2 (Contêineres), Nível 3 (Componentes)
3. **Modelo de Dados Conceitual** – entidades por serviço e relacionamentos entre serviços
4. **Estrutura de Mensageria do Kafka** – tópicos, catálogo de eventos, chaves, consumidores, Outbox e idempotência
5. **Pipeline de Desenvolvimento** – CI/CD, Docker Compose e portões de qualidade
6. **Processo de Gerenciamento da Configuração de Software (GCS)** – ferramenta, branches e regras
7. **Rastreabilidade com o enunciado**
8. **Atualizações do Pacote 3** – o que mudou entre o desenho e a implementação

1. Visão geral e decisões arquiteturais

Este documento detalha a arquitetura da plataforma PulsePass definida nos requisitos do Pacote 1 (histórias US01–US33 e requisitos não funcionais RNF01–RNF23). A solução é composta por microsserviços orientados a eventos, com um banco de dados por serviço, Kafka como barramento e comunicação síncrona apenas na borda (SPA → API Gateway → serviço) e nas chamadas ao gateway de pagamento simulado.

1.1 Premissas e stack

Camada	Escolha	Motivo
Backend	Java 21, Spring Boot 3, Spring Cloud Gateway, Resilience4j, Maven	Alinhado ao conteúdo da disciplina (Spring Cloud, JUnit 5, Testcontainers, PIT).
Frontend	SPA responsiva em TypeScript, servida por Nginx	RNF21/RNF22; sem aplicativo nativo (fora do escopo).
Identidade	Keycloak (OIDC, JWT RS256)	RNF12; senhas existem somente no IdP (RNF15).
Persistência	PostgreSQL 16, um banco lógico por serviço	Transações ACID para estados, Outbox e idempotência.
Cache e coordenação	Redis 7	Cache do catálogo (RNF06), filtro de contenção de assentos e fila virtual (RNF23).
Streaming	Apache Kafka (modo KRaft) e Kafka UI	Desacoplamento e reprocessamento (RNF04, RNF05, RNF11).
Tempo real	WebSocket/STOMP em serviço dedicado	Mapa de assentos, status do pedido e painel (US14, US27, US32).
Observabilidade	OpenTelemetry, Prometheus, Grafana, logs JSON com `correlation-id`	RNF16.

1.2 Decisões arquiteturais (ADRs)

ADR	Decisão	Justificativa e consequências
001	Banco de dados por serviço; nenhuma chave estrangeira entre serviços	Autonomia de implantação e escala. Referências entre serviços são apenas identificadores; a consistência é eventual e feita por eventos.
002	Reserva de assento com `UPDATE` condicional no PostgreSQL como fonte da verdade, e Redis (`SET NX PX`) como filtro de contenção	Garante zero sobre-venda (RNF01) mesmo se o Redis falhar ou perder a chave: quem decide é a contagem de linhas afetadas por `UPDATE ... WHERE status = 'LIVRE'` . O Redis só reduz a carga no banco durante o pico. A capacidade de um lote segue a mesma regra, com contadores atualizados por `UPDATE` condicional.
003	Transactional Outbox com publicador por polling (`FOR UPDATE SKIP LOCKED`)	Elimina a escrita dupla banco+Kafka (RNF04). Publicação é *at-least-once*. Alternativa futura: CDC com Debezium, sem mudar o contrato dos eventos.
004	Consumidores idempotentes com tabela `evento_processado` gravada na mesma transação do efeito	Reprocessar um evento não duplica ingressos nem transições (RNF05).

ADR	Decisão	Justificativa e consequências
005	API Gateway (Spring Cloud Gateway) valida o JWT e aplica rate limit; os serviços revalidam o papel (RBAC)	Ponto único de entrada (RNF12, RNF13); defesa em profundidade.
006	`realtime-service` separado, sem estado e sem banco	Escala independente das conexões WebSocket (RNF09). Cada réplica usa um *consumer group* próprio para receber todos os eventos e entregar às suas sessões.
007	Gateway de pagamento simulado como contêiner próprio, com aprovação, recusa e atraso configuráveis	Permite testar timeout, retentativa e circuit breaker (RNF10).
008	Expiração de reserva executada pelo Inventário (worker agendado), com suspensão enquanto o pagamento estiver em curso	Evita expirar uma reserva cujo pagamento está sendo processado; a suspensão é limitada a 30 s, respeitando a tolerância do RNF02 (10 min + 30 s).

1.3 Rastreabilidade dos requisitos não funcionais

RNF	Como a arquitetura atende
RNF01, RNF02	ADR-002 e ADR-008; reserva ativa com `expira_em`; worker de expiração e evento `ReservaExpirada`.
RNF03	Filtro de `Idempotency-Key` e tabela `chave_idempotencia` no Checkout (seção 2.3.6).
RNF04, RNF05	ADR-003 e ADR-004; seção 4.5.
RNF06 a RNF08	Catálogo lido do cache Redis (RNF06); reserva com filtro Redis antes do banco (RNF07); emissão assíncrona, com o tempo entre `PedidoPago` e `IngressoEmitido` medido pelo telemetria-service (RNF08). Testes de carga com k6 ficam como bônus (seção 5.2).
RNF09, RNF11	Serviços sem estado em memória; partições por chave; mensagens retidas no Kafka enquanto o emissao-service (emissão e notificações) estiver fora.
RNF10	`GatewayPagamentoClient` com timeout, retentativa com backoff e circuit breaker (Resilience4j).
RNF12–RNF15	Keycloak, API Gateway, RBAC nos serviços, QR assinado pela Emissão e dados pessoais apenas no IdP (os serviços guardam somente `usuario_id`).
RNF16	OpenTelemetry Collector, Prometheus e Grafana no Compose; `traceparent` propagado em HTTP e nos cabeçalhos Kafka.
RNF17–RNF20	Pipeline e GCS (seções 5 e 6).
RNF21–RNF23	SPA responsiva e acessível (contêiner SPA); fila virtual por ordem de chegada no Inventário (Figura 3 e seção 2.3.6).

1.4 Limitações assumidas

- **Kafka com um único broker** no Compose (replicação 1); o desenho já parametriza o fator de replicação.
- **Pagamento simulado:** aprovação tardia após *timeout* é tratada como estorno simulado, sem integração real com adquirente.
- **Telemetria** aceita perda eventual de eventos de navegação (não críticos); eventos de domínio nunca se perdem, pelo Outbox.
- **Sem multirregião nem alta disponibilidade do PostgreSQL**: fora do escopo do projeto acadêmico.

2. Arquitetura no Modelo C4

A arquitetura é descrita em três níveis. Os diagramas usam a notação C4 (pessoa, sistema, contêiner, componente), com tecnologia entre colchetes. Linhas tracejadas indicam comunicação assíncrona, indireta ou de telemetria.

2.1 Nível 1 – Diagrama de Contexto

O PulsePass atende quatro perfis de usuário e depende de dois sistemas externos. As fronteiras do software são: tudo o que o grupo constrói e implanta está dentro do sistema PulsePass; o gateway de pagamento (simulado) e o serviço de e-mail ficam fora.

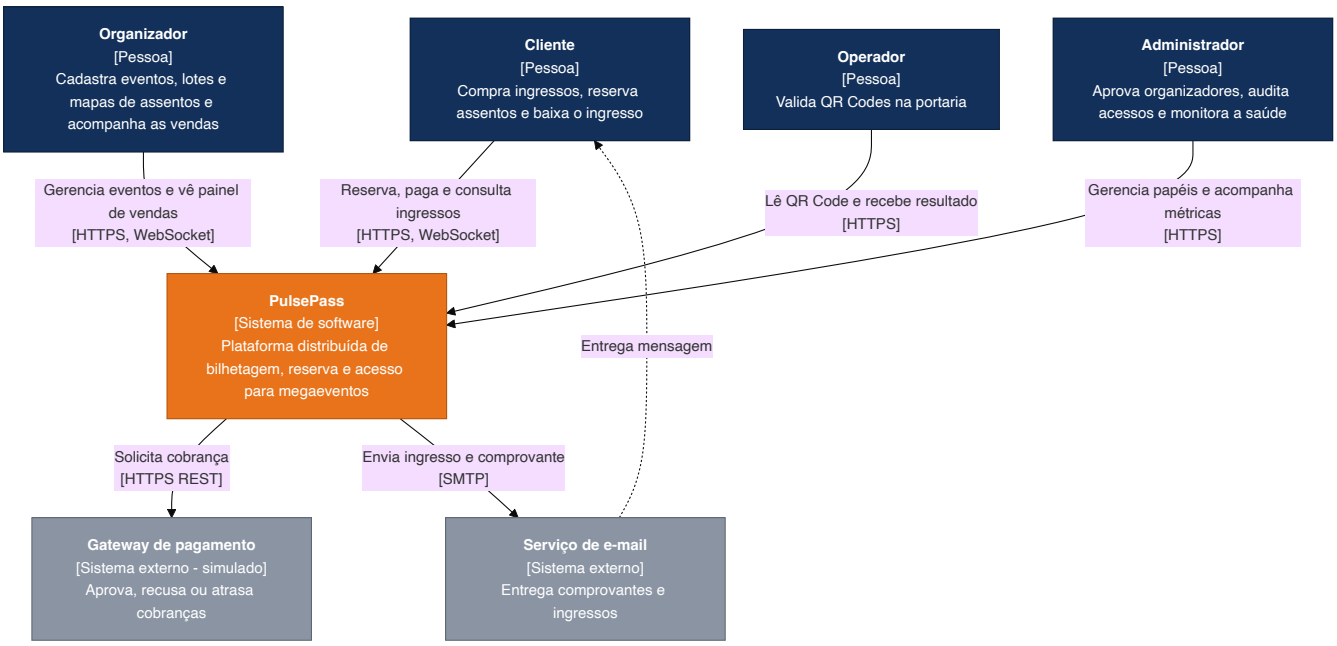


Figura 1 – C4 Nível 1: contexto do PulsePass.

2.2 Nível 2 – Diagrama de Contêineres

Distribuição física: SPA, API Gateway, Keycloak, cinco serviços de negócio, o serviço de tempo real, bancos, Redis, cluster Kafka e a stack de observabilidade. Em desenvolvimento, todos sobem por `docker compose up`; os cinco bancos lógicos podem residir em uma única instância PostgreSQL.

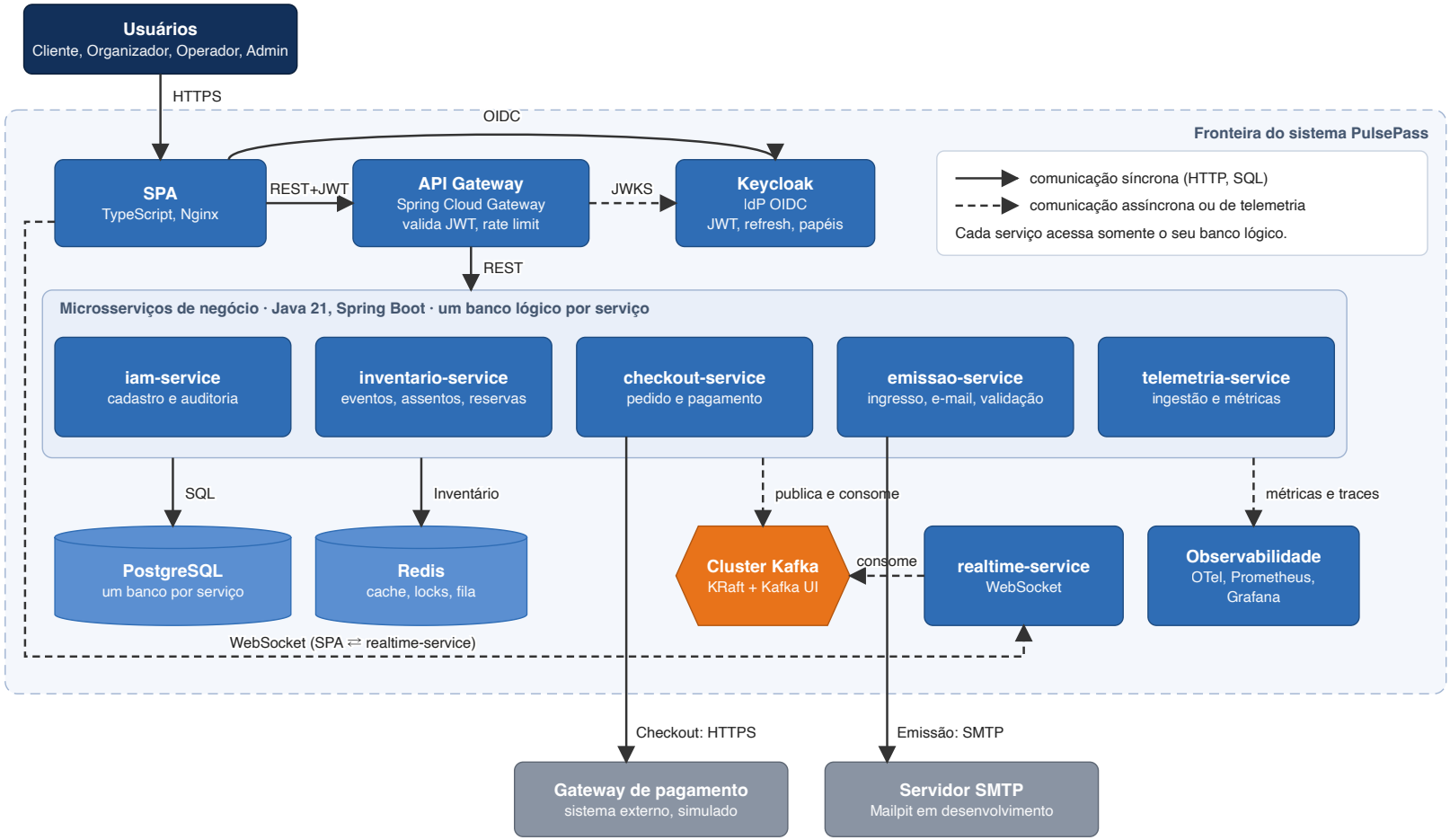


Figura 2 – C4 Nível 2: contêineres. Linhas tracejadas: comunicação assíncrona ou de telemetria. Fronteira tracejada: tudo o que o grupo implanta (ADR-001).

Responsabilidades dos contêineres

Contêiner	Responsabilidade	Dados e histórias
SPA	Catálogo, mapa de assentos, checkout com contagem regressiva sincronizada pelo servidor, meus ingressos, painéis e leitor de QR.	US11–US14, US19, US26, US28
API Gateway	Entrada única; valida assinatura e expiração do JWT (401); roteia por prefixo; rate limit por IP e por usuário.	RNF12
Keycloak	Cadastro, login, emissão de access token (15 min) e refresh token (7 dias), papéis Cliente, Organizador, Operador e Admin.	US01–US04
iam-service	Cadastro e aprovação de organizadores (altera papéis pela Admin API do Keycloak); trilha de auditoria de credenciais, alimentada por eventos do Keycloak.	US05, US06
inventario-service	Eventos, lotes e mapas; catálogo com cache Redis; reserva atômica, cancelamento e expiração; fila virtual.	US07–US18
checkout-service	Máquina de estados do pedido; pagamento idempotente; cliente do gateway com resiliência; Outbox.	US19–US23
emissao-service	Consumo de `PedidoPago`; ingresso em PDF com QR assinado e carimbo de tempo; e-mail; validação na portaria.	US24–US29
telemetria-service	Ingestão de eventos de navegação e compra; agregação por janela; métricas operacionais.	US31–US33
realtime-service	Mantém sessões WebSocket autenticadas (valida o JWT no handshake, pelo JWKS do Keycloak) e entrega atualizações vindas do Kafka (assentos, pedido, painel), filtrando mensagens de pedido e ingresso pelo `clienteId` do usuário.	US14, US27, US32

2.3 Nível 3 – Diagramas de Componentes

Os cinco serviços de negócio são detalhados abaixo, começando pelos de maior complexidade técnica (Inventário, Checkout e Emissão). Todos seguem a mesma divisão em camadas: **controladores de borda** (REST), **casos de uso** (aplicação), **domínio** (regras e máquinas de estado, sem dependência de framework), **repositórios e adaptadores** (JPA, Redis, HTTP, SMTP), **workers do Outbox** e **consumidores de mensageria**.

2.3.1 inventario-service

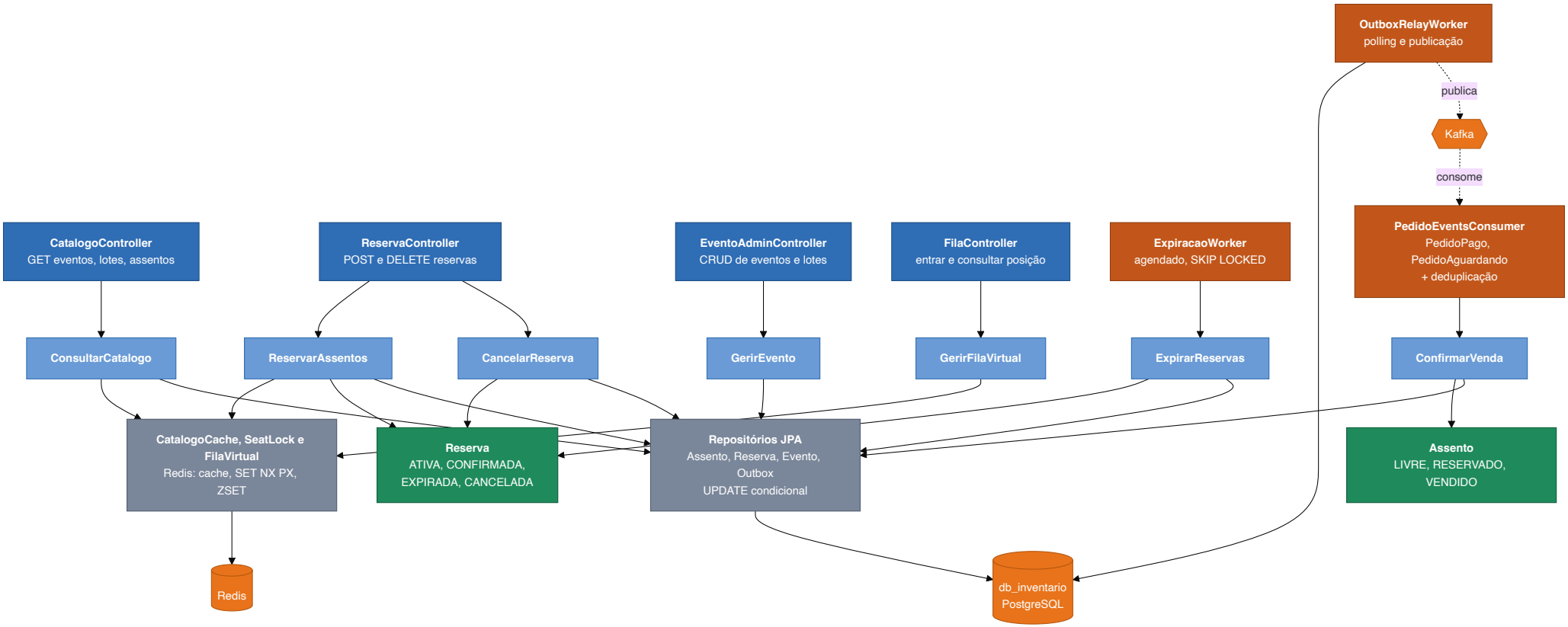


Figura 3 – C4 Nível 3: componentes do inventario-service. Cores: azul, controladores de borda; azul claro, casos de uso; verde, domínio; cinza, adaptadores; laranja escuro, workers e consumidores; laranja, infraestrutura. Em todos, o `JwtAuthFilter` (resource server do Keycloak) protege os controladores com RBAC.

2.3.2 checkout-service

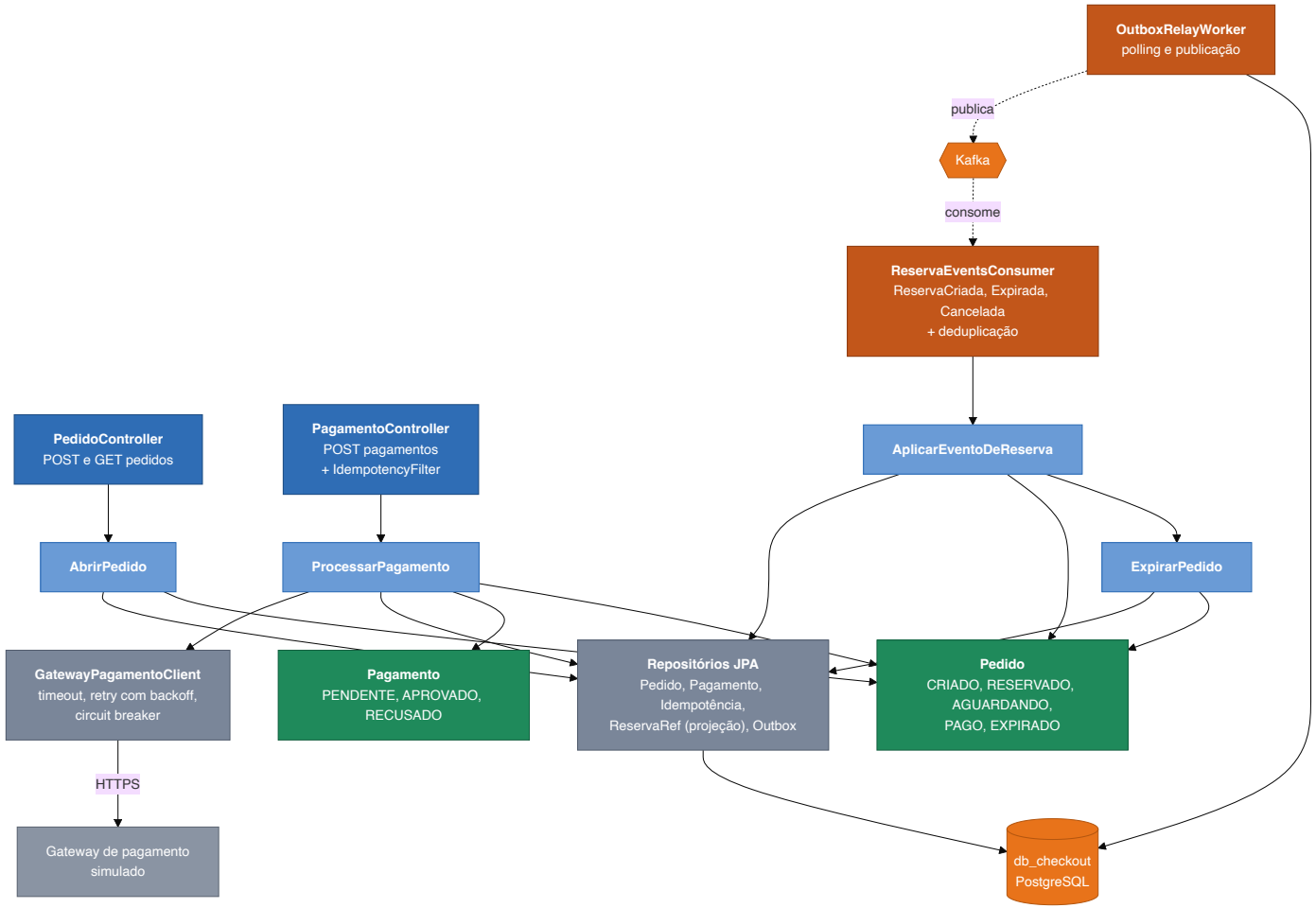


Figura 4 – C4 Nível 3: componentes do checkout-service (mesma legenda de cores da Figura 3).

2.3.3 emissao-service

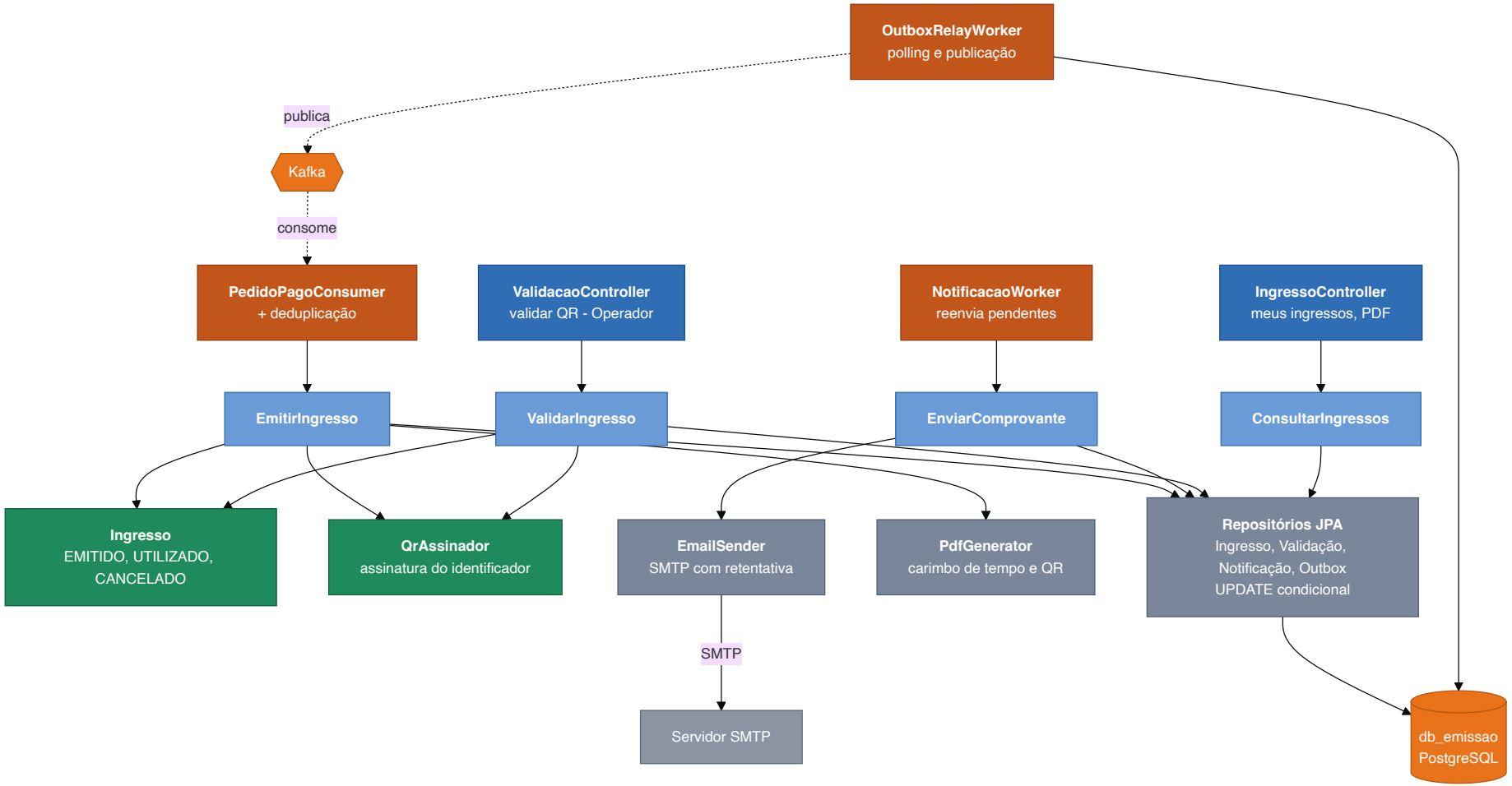


Figura 5 – C4 Nível 3: componentes do emissao-service (mesma legenda de cores da Figura 3).

2.3.4 iam-service

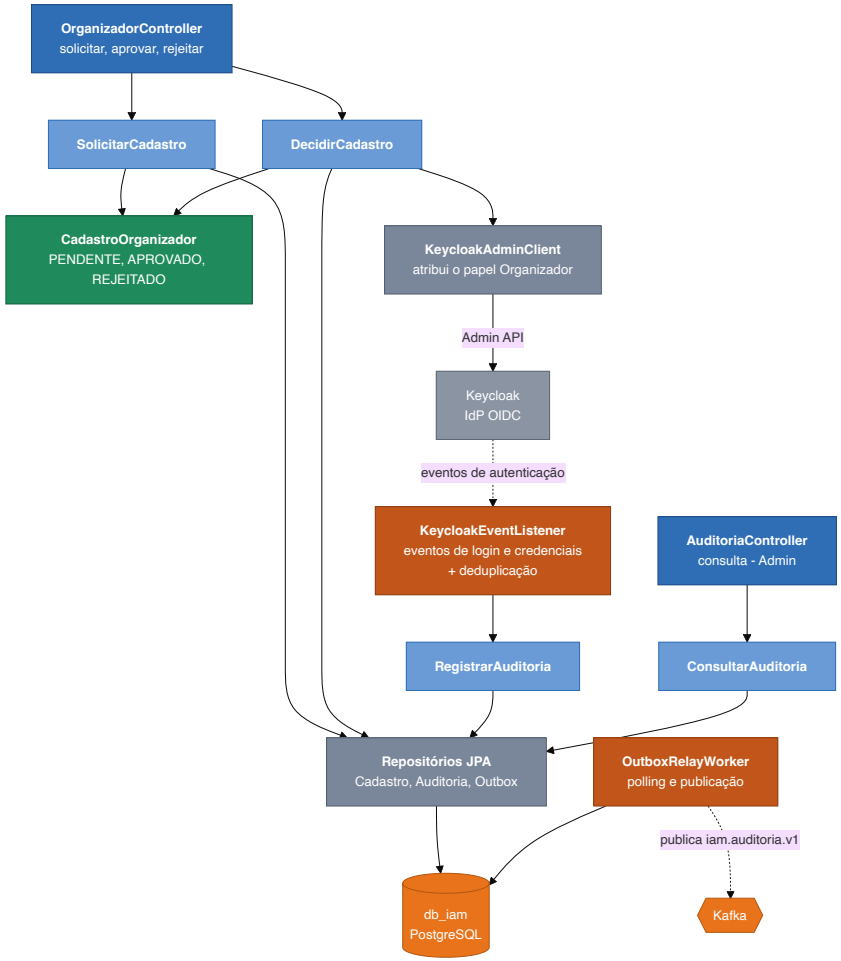


Figura 6 – C4 Nível 3: componentes do iam-service (mesma legenda de cores da Figura 3).

2.3.5 telemetria-service

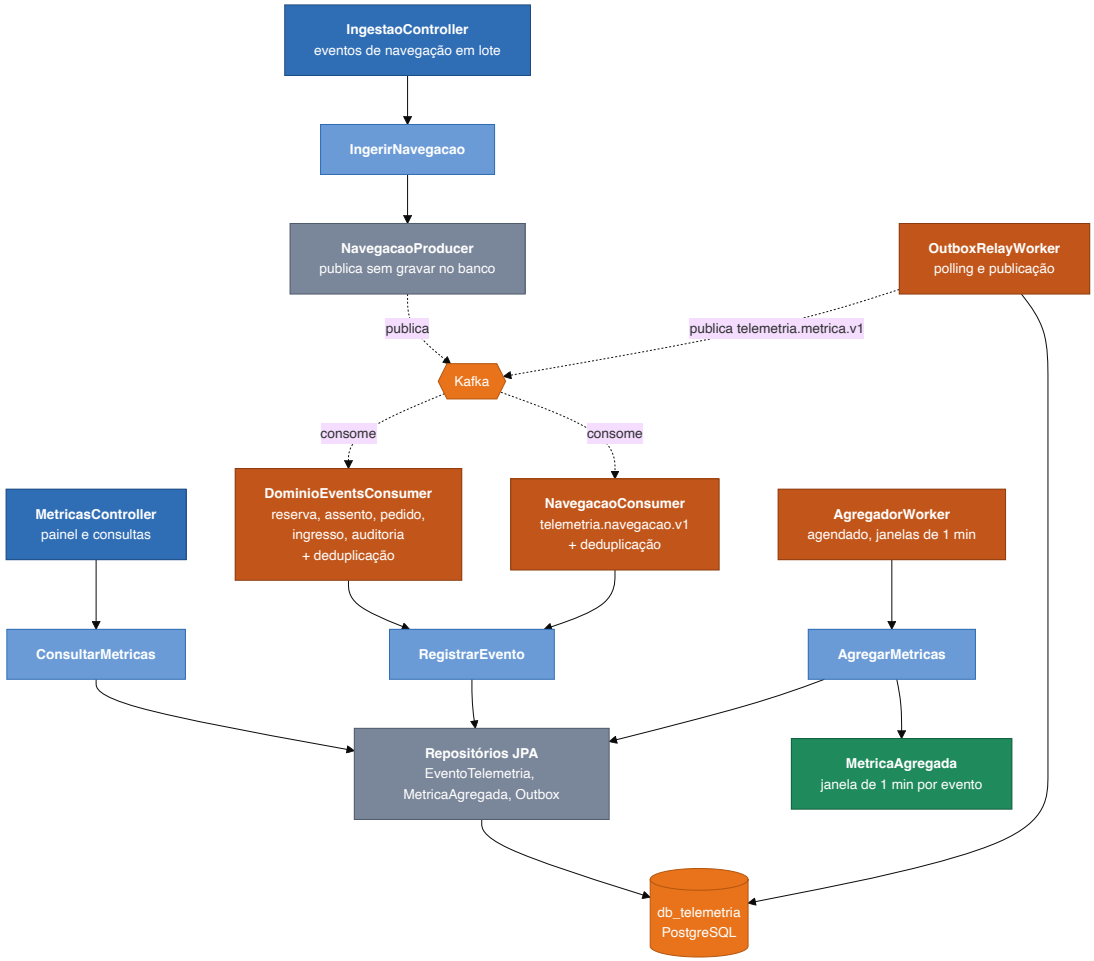


Figura 7 – C4 Nível 3: componentes do telemetria-service (mesma legenda de cores da Figura 3).

2.3.6 Comportamento dos componentes críticos

Fluxo crítico de `ReservarAssentos` (US15). (1) tenta `SET NX PX` no Redis para cada assento, descartando rápido quem perdeu a disputa; (2) abre a transação e executa `UPDATE assento SET status='RESERVADO' WHERE id=? AND status='LIVRE'`; se alguma contagem for 0, desfaz tudo, libera as chaves do Redis e responde HTTP 409; (3) cria a `reserva` ATIVA com `expira_em = agora + 10 min` e grava `ReservaCriada` e `AssentoReservado` no Outbox, na mesma transação; (4) após o commit, o relay publica. Para lotes sem assento marcado, o passo (2) é `UPDATE lote SET qtd_reservada = qtd_reservada + :n WHERE id=? AND qtd_reservada + qtd_vendida + :n <= quantidade`, com a mesma regra de contagem de linhas afetadas; expiração e cancelamento devolvem as unidades, e `PedidoPago` converte `qtd_reservada` em `qtd_vendida`. O `ExpiracaoWorker` seleciona reservas vencidas com `FOR UPDATE SKIP LOCKED`, libera os assentos e grava `ReservaExpirada` no Outbox; ele ignora reservas cujo pedido está em pagamento por até 30 s (ADR-008).

Máquina de estados do pedido. `CRIADO`: pedido aberto na tela de revisão (`POST /pedidos` com `reservaId`, US19). `RESERVADO`: o Checkout confirmou, pela projeção alimentada por `ReservaCriada`, que a reserva está ativa; fixa o total e o `expira_em`. Se o evento ainda não chegou, o pedido permanece em `CRIADO` e avança quando chegar. `AGUARDANDO`: pagamento aceito e enviado ao gateway. `PAGO`: gateway aprovou. `EXPIRADO`: reserva expirada ou cancelada pelo cliente (o motivo é registrado). Se o gateway recusar o pagamento, o pedido volta a `RESERVADO` e o cliente pode tentar de novo enquanto a reserva estiver ativa (US23). O payload de `ReservaCriada` traz os itens (assento ou lote, quantidade e preço unitário), que o Checkout guarda em `RESERVA_REF` para fixar o total sem consulta síncrona. Transições inválidas lançam exceção de domínio e são cobertas pelos testes unitários (meta ≥ 70%).

Pagamento idempotente (US21, RNF03). O `IdempotencyFilter` procura a chave em `chave_idempotencia`. Se existir com o mesmo hash de requisição, devolve a resposta guardada, sem chamar o gateway; se o hash for diferente, responde 422. Se não existir: **TX1** insere a chave (índice único), o pagamento PENDENTE e move o pedido para AGUARDANDO; a chamada ao gateway ocorre **fora** de transação; **TX2** atualiza o pagamento e o pedido, grava `PedidoPago` (ou `PagamentoRecusado`) no Outbox e guarda a resposta na chave. O timeout do gateway (20 s) é menor que a suspensão da expiração (30 s, ADR-008): a resposta, aprovada ou recusada, chega antes de a reserva poder expirar. Se o gateway estourar o timeout, o pedido volta a RESERVADO; uma aprovação tardia vira estorno simulado (limitação registrada em 1.4).

Emissão e validação. `EmitirIngresso` deduplica pelo `event_id`, cria o ingresso, assina o identificador (`QrAssinador`, RNF14), gera o PDF com carimbo de tempo e grava `IngressoEmitido` no Outbox. O envio de e-mail é feito pelo `NotificacaoWorker`, fora do caminho crítico: falha de SMTP não impede a emissão (RNF11). `ValidarIngresso` verifica a assinatura e executa `UPDATE ingresso SET status='UTILIZADO' WHERE id=? AND status='EMITIDO'`; com 0 linhas afetadas responde "Ingresso já utilizado" com o horário da primeira leitura (US28, US29).

Fila virtual (US18, RNF23). Quando a demanda excede a capacidade configurada para o evento, o `FilaController` registra o cliente em um ZSET do Redis por evento, ordenado pelo instante de chegada. O cliente consulta sua posição e, quando chega a vez, recebe um token de acesso de curta duração, exigido por `POST /reservas`. A liberação segue a ordem de chegada, em lotes.

Cadastro de organizador (US05, US06). `SolicitarCadastro` grava o pedido como PENDENTE. `DecidirCadastro`, restrito ao Admin, chama a Admin API do Keycloak para atribuir o papel Organizador (operação idempotente) e só então confirma APROVADO no banco, junto com o registro de auditoria e o evento `OrganizadorAprovado` no Outbox. Se o Keycloak falhar, o cadastro continua PENDENTE e a decisão pode ser repetida.

Telemetria (US31 a US33). A SPA envia eventos de navegação em lote; o `NavegacaoProducer` os publica em `telemetria.navegacao.v1` sem gravar no banco, portanto sem escrita dupla. Os consumidores gravam `EVENTO_TELEMETRIA` com deduplicação por `eventId`; o `AgregadorWorker` calcula janelas de 1 min, grava `METRICA_AGREGADA` e o evento `MetricaAgregada` no Outbox, na mesma transação. Métricas operacionais (latência, erros, lag do Kafka) vêm do Prometheus e do Grafana.

3. Modelo de Dados Conceitual

O modelo é conceitual: mostra entidades, atributos principais e relacionamentos, sem tipos físicos, índices ou particionamento. Em conformidade com a ADR-001, cada serviço é dono dos seus dados; **relacionamentos entre serviços aparecem como atributos de referência** (identificador) e não como chaves estrangeiras. Usuários, senhas e papéis residem somente no Keycloak (RNF15) e, nos demais serviços, aparecem apenas como `usuario_id` (o `sub` do JWT). Todo serviço que publica ou consome eventos (Inventário, Checkout, Emissão, IAM e Telemetria) mantém ainda duas tabelas técnicas, omitidas dos diagramas por serem iguais: `OUTBOX_EVENT` (`id`, `tipo`, `payload` JSON, `publicado_em`) e `EVENTO_PROCESSADO` (`event_id` como chave primária, `processado_em`).

3.1 Visão integrada

A Figura 8 reúne as entidades principais de todos os serviços. Cada caixa maior é o banco de um serviço; setas contínuas são relações dentro do mesmo serviço, e **setas tracejadas atravessam a fronteira entre serviços**, mantidas apenas por identificador e por eventos (ADR-001). Os usuários e seus papéis vivem no Keycloak, não em tabela de serviço de negócio. Os atributos de cada entidade estão nas Figuras 9 a 11.

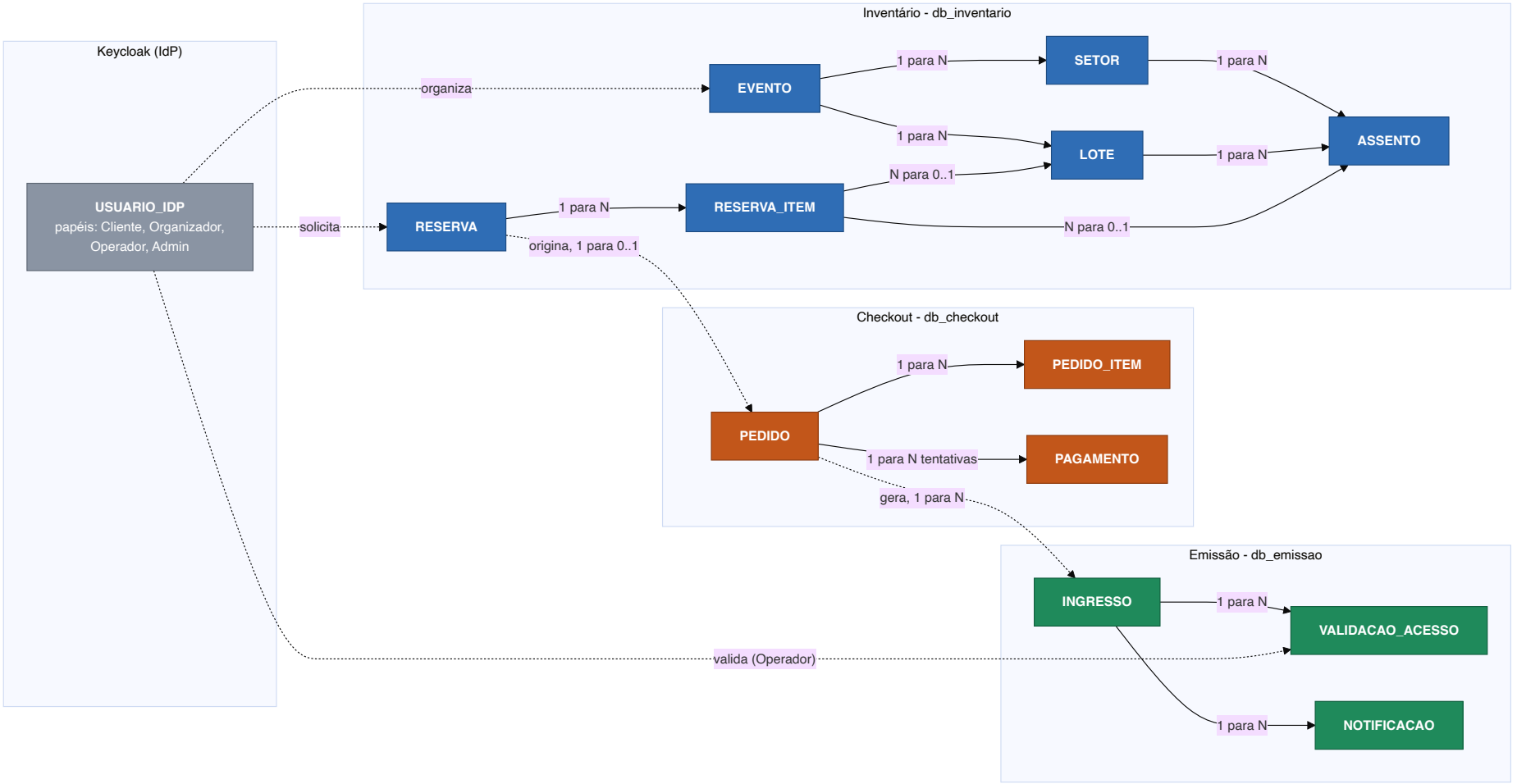


Figura 8 – Modelo de dados conceitual integrado, com as fronteiras entre serviços.

3.2 Referências entre serviços e telemetria

O modelo conceitual do **telemetria-service** (`db_telemetria`) tem duas entidades: `EVENTO_TELEMETRIA` (`id` , `tipo` , `evento_id` com referência ao Inventário, `sessao_id` , `atributos` em JSON, `ocorrido_em`) e `METRICA_AGREGADA` (chave composta por `evento_id` , `metrica` e `janela_inicio` , mais `valor`). Elas não se relacionam por chave, porque a agregação é derivada. O **realtime-service** não possui banco.

Origem	Referência	Como é mantida
Checkout `PEDIDO.reserva_id`	Inventário `RESERVA`	Projeção `RESERVA_REF` atualizada por `ReservaCriada` , `ReservaExpirada` , `ReservaCancelada` .
Inventário `ASSENTO.status`	Checkout `PEDIDO`	Evento `PedidoPago` converte assentos RESERVADO em VENDIDO e reserva em CONFIRMADA.
Emissão `INGRESSO.pedido_id`	Checkout `PEDIDO`	Preenchido a partir do payload de `PedidoPago` , que carrega os itens necessários (sem consulta síncrona).
Qualquer `usuario_id`	Keycloak (`sub`)	Vem do JWT; nenhum dado pessoal é replicado (RNF15).

3.3 Inventário

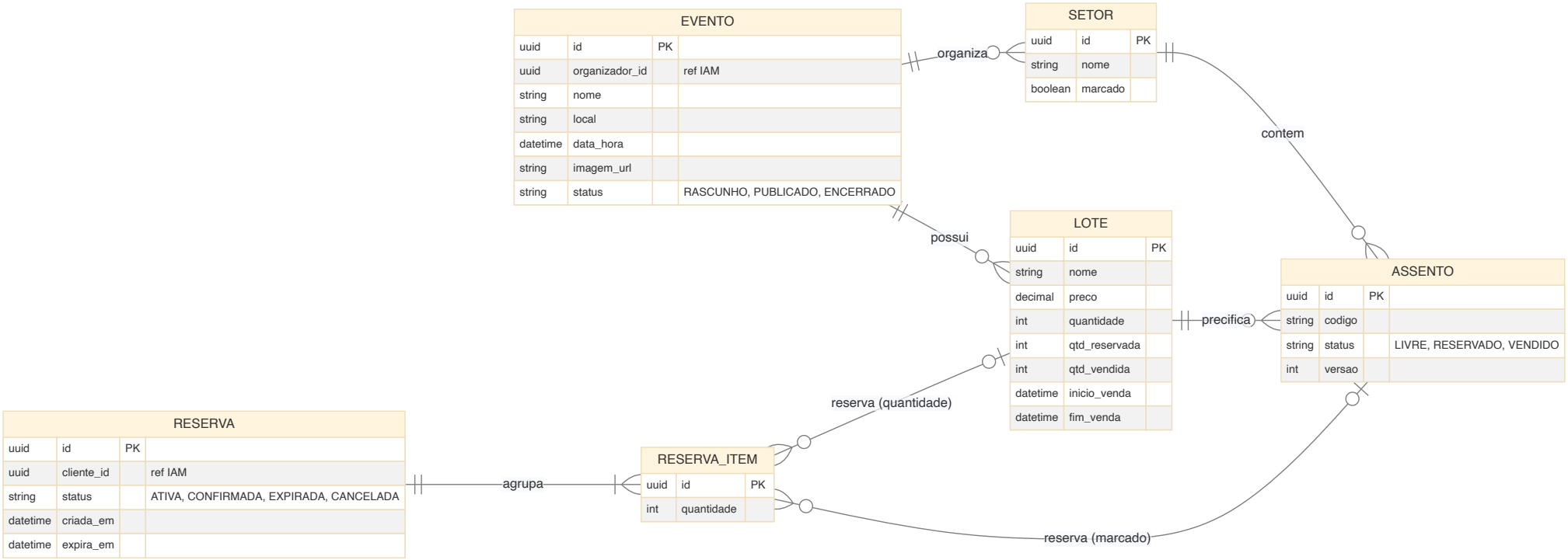


Figura 9 – Modelo conceitual do Inventário. Ingressos sem assento marcado usam `RESERVA_ITEM.quantidade` contra o `LOTE`.

3.4 Checkout

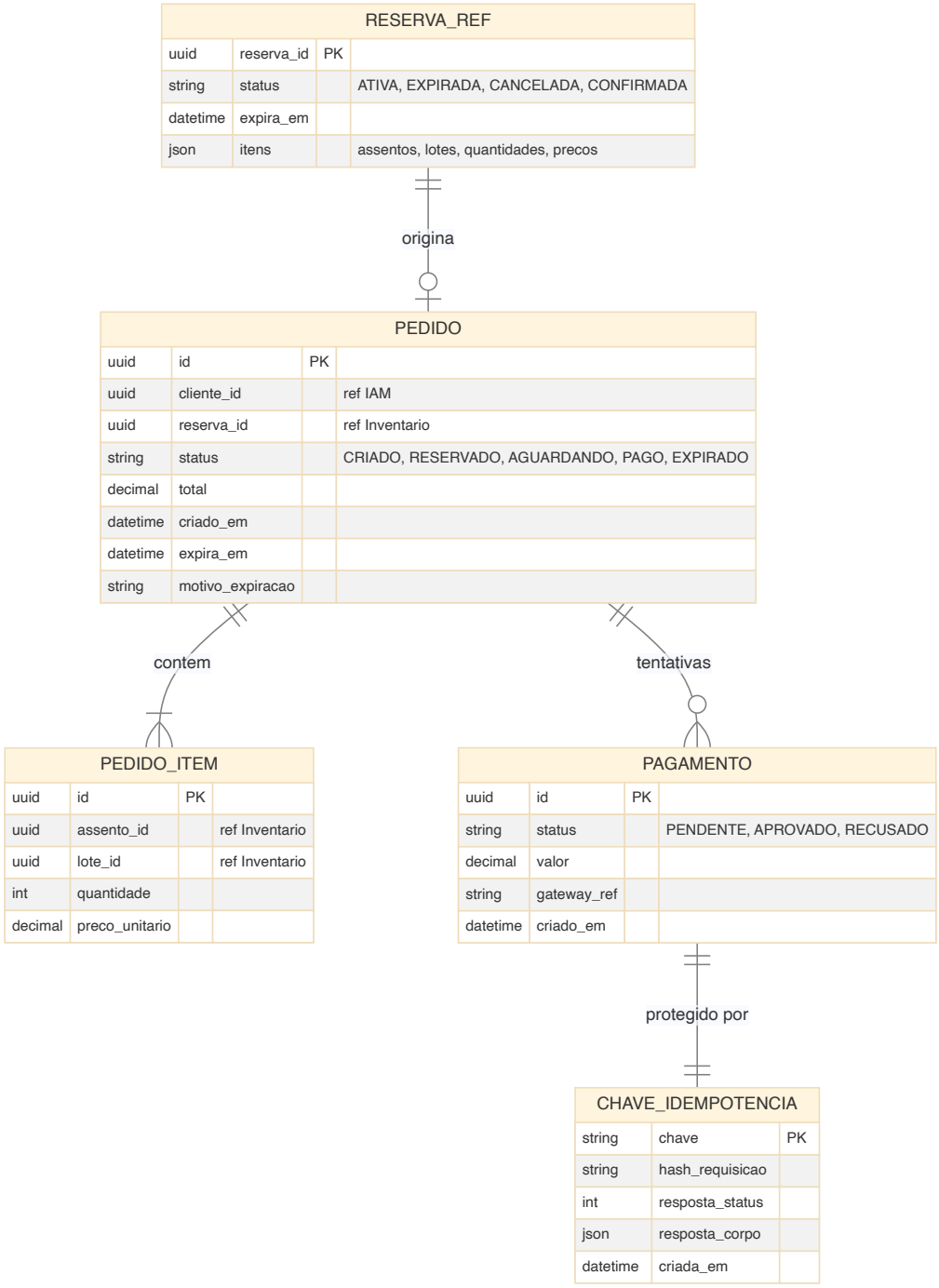


Figura 10 – Modelo conceitual do Checkout. `RESERVA_REF` é uma projeção local alimentada por eventos do Inventário, com os itens e preços recebidos em `ReservaCriada`.

3.5 Emissão e IAM

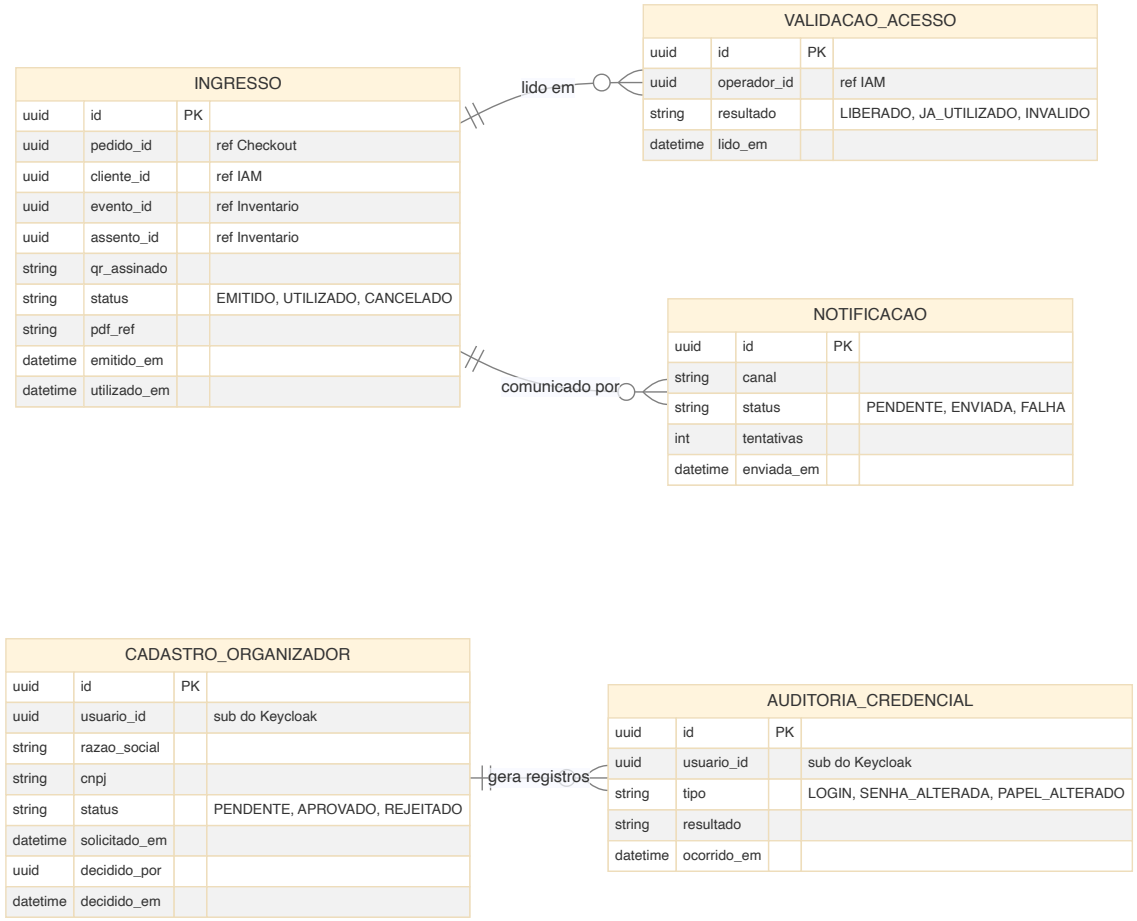


Figura 11 – Modelo conceitual da Emissão (`db_emissao`) e do IAM (`db_iam`). Cada serviço também mantém `OUTBOX_EVENT` e `EVENTO_PROCESSADO` .

4. Estrutura de Mensageria do Kafka

4.1 Convenções

- **Nome do tópico:** `<servico>.<agregado>.v1`. Um tópico por agregado, com vários tipos de evento; o sufixo `v1` versiona o contrato.
- **Chave da mensagem:** identificador do agregado, garantindo ordem por agregado dentro de uma partição.
- **Envelope:** `eventId` (UUID), `eventType`, `occurredAt`, `aggregateId`, `version`, `payload`. Cabeçalhos Kafka: `correlation-id` e `traceparent` (RNF16).
- **Formato:** JSON com schema documentado em `/specs/<modulo>/<feature>/design.md`. Regra de compatibilidade: somente adição de campos opcionais dentro de `v1`.
- **Produtor:** `acks=all`, `enable.idempotence=true`. **Consumidor:** `enable.auto.commit=false`, *commit* do offset após a transação local.

4.2 Tópicos

Tópico	Eventos	Produtor	Consumidores	Chave	Partições e retenção
`inventario.reserva.v1`	ReservaCriada, ReservaExpirada, ReservaCancelada	Inventário	Checkout, Telemetria	reservaId	6 partições, 7 dias
`inventario.assento.v1`	AssentoReservado, AssentoLiberado, AssentoVendido	Inventário	Realtime, Telemetria	eventoId	6 partições, 1 dia
`checkout.pedido.v1`	PedidoCriado, PedidoReservado, PedidoAguardandoPagamento, PedidoPago, PagamentoRecusado, PedidoExpirado	Checkout	Inventário, Emissão, Realtime, Telemetria	pedidoId	6 partições, 7 dias
`emissao.ingresso.v1`	IngressoEmitido, IngressoUtilizado, ComprovanteEnviado	Emissão	Realtime, Telemetria	pedidoId	3 partições, 7 dias
`iam.auditoria.v1`	LoginRealizado, SenhaAlterada, PapelAlterado, OrganizadorAprovado	IAM	Telemetria	usuarioId	3 partições, 30 dias
`telemetria.navegacao.v1`	PaginaVisualizada, AssentoSelecionado, CheckoutIniciado	Telemetria (ingestão)	Telemetria (agregador)	sessaoId	6 partições, 1 dia
`telemetria.metrica.v1`	MetricaAgregada	Telemetria	Realtime	eventoId	3 partições, 1 dia
`<topico>.<grupo>.dlq`	Mensagem original e causa	Consumidor	Operação (Kafka UI)	mantida	1 partição, 14 dias

No Compose há um único broker, portanto fator de replicação 1; o valor é parametrizado por variável de ambiente para 3 em um cluster maior.

4.3 Catálogo de eventos e payloads principais

Evento	Tópico	Campos principais do payload
ReservaCriada	`inventario.reserva.v1`	`reservaId`, `clienteId`, `eventoId`, `itens[]` (`assentoId` ou `loteId`, `quantidade`, `precoUnitario`), `criadaEm`, `expiraEm`
ReservaExpirada, ReservaCancelada	`inventario.reserva.v1`	`reservaId`, `motivo`, `ocorridaEm`
AssentoReservado, AssentoLiberado, AssentoVendido	`inventario.assento.v1`	`eventoId`, `assentoId`, `status`
PedidoReservado, PedidoAguardandoPagamento	`checkout.pedido.v1`	`pedidoId`, `reservaId`, `total`, `expiraEm`
PedidoPago	`checkout.pedido.v1`	`pedidoId`, `clienteId`, `eventoId`, `itens[]`, `total`, `pagoEm`
PagamentoRecusado, PedidoExpirado	`checkout.pedido.v1`	`pedidoId`, `motivo` (por exemplo `CANCELADO_PELO_CLIENTE`), `ocorridoEm`
IngressoEmitido	`emissao.ingresso.v1`	`ingressoId`, `pedidoId`, `clienteId`, `eventoId`, `assentoId` ou `loteId`, `emitidoEm`
IngressoUtilizado	`emissao.ingresso.v1`	`ingressoId`, `operadorId`, `lidoEm`
OrganizadorAprovado	`iam.auditoria.v1`	`usuarioId`, `decididoPor`, `decididoEm`
MetricaAgregada	`telemetria.metrica.v1`	`eventoId`, `metrica`, `janelaInicio`, `valor`

Exemplo de mensagem (`PedidoPago`, chave `pedidoId`):

```
{
  "eventId": "7c1e5a52-3f0b-4d57-9b0e-2a64f1d1c8aa",
  "eventType": "PedidoPago",
  "occurredAt": "2026-11-04T21:15:03Z",
  "aggregateId": "b3d1c2f0-91a7-4c53-8a1e-5d2f7e9c0a11",
  "version": 1,
  "payload": {
    "pedidoId": "b3d1c2f0-91a7-4c53-8a1e-5d2f7e9c0a11",
    "clienteId": "e4f6a9d2-0c7b-4e1a-b5d3-8f2a1c9e7b40",
    "eventoId": "a91f0c77-2d4e-4b86-9c13-6e8d5b2f1a30",
    "itens": [ { "assentoId": "3f2b8c1d-6a95-4e07-a1d4-9b7c0e5f2a68", "quantidade": 1,
"precoUnitario": 250.00 } ],
    "total": 250.00,
    "pagoEm": "2026-11-04T21:15:03Z"
  }
}
```

4.4 Mapa produtores, tópicos e consumidores

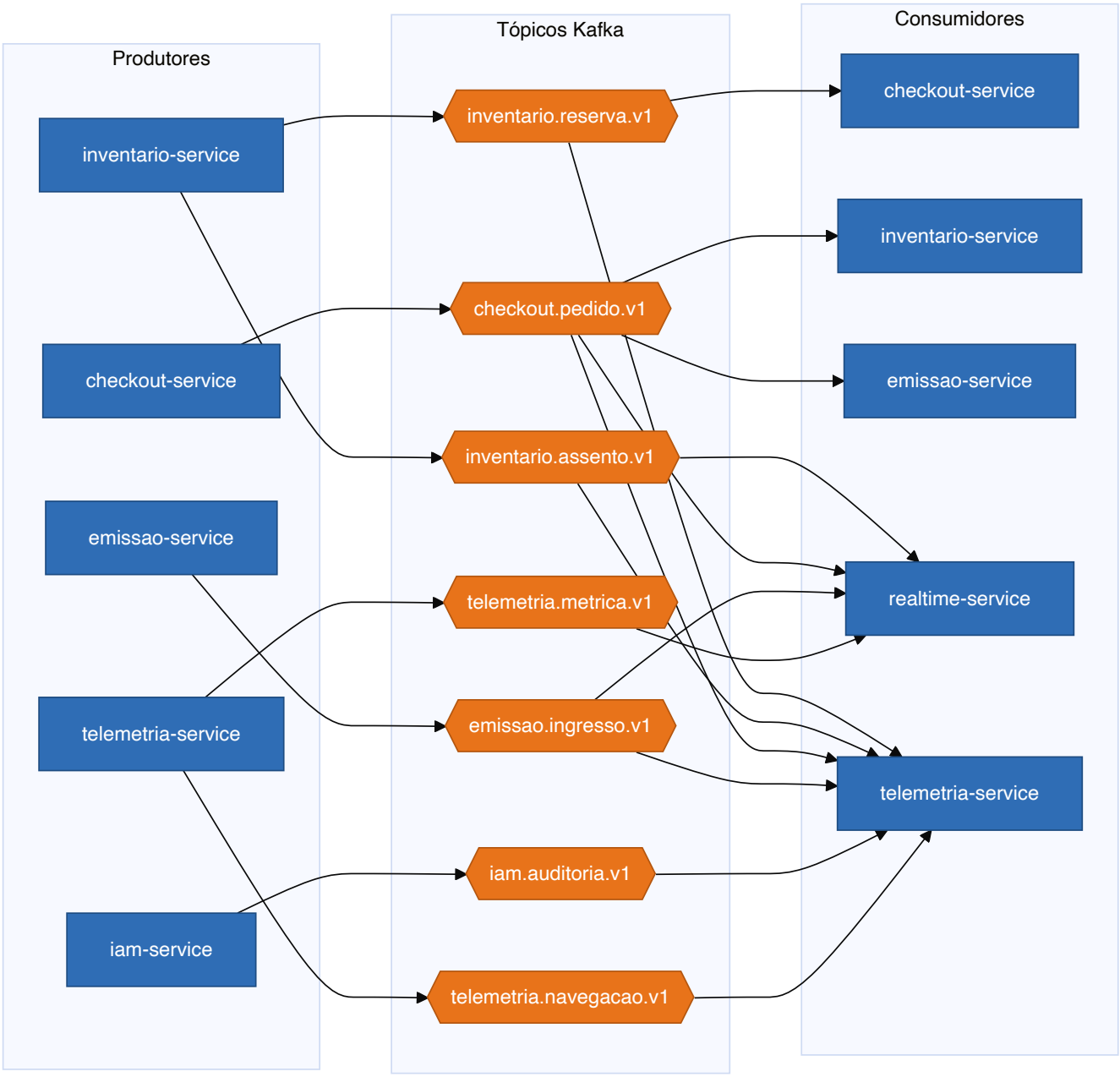


Figura 12 – Fluxo de eventos entre serviços (seta para o tópico: publica; seta do tópico: consome).

Consumer groups

Serviço	Group ID	Observação
checkout, inventario, emissao, telemetria	`<servico>-<topico>`	Réplicas do mesmo serviço dividem as partições (RNF09).
realtime-service	`realtime-<instanceId>`	Um grupo por réplica: toda réplica recebe todos os eventos e entrega apenas às sessões WebSocket que ela mantém.

4.5 Entrega confiável: Outbox, idempotência e falhas

- **Escrita dupla (Dual-Write).** Nenhum serviço grava no banco e publica no Kafka em duas operações independentes. O serviço grava o estado e a linha em `outbox_event` **na mesma transação ACID**. O `OutboxRelayWorker` lê linhas não publicadas com `FOR UPDATE SKIP LOCKED` (várias réplicas sem conflito),

publica com produtor idempotente e marca `publicado_em`. Se o Kafka estiver fora, as linhas ficam pendentes e saem quando ele voltar (RNF04).

- **Sistemas externos fora do Kafka.** O Checkout chama o gateway de pagamento fora da transação e protegido por idempotência (seção 2.3.6); o IAM chama a Admin API do Keycloak antes de confirmar a decisão, em operação idempotente que pode ser repetida. Assim, nenhuma chamada externa deixa o banco e o mundo exterior inconsistentes sem caminho de reprocessamento.
- **Entrega *at-least-once* e idempotência no consumo.** O relay pode republicar após uma falha entre o envio e a marcação. Por isso todo consumidor insere o `eventId` em `evento_processado` (chave primária) **na mesma transação** do efeito. Violação de chave significa evento já tratado: o consumidor confirma o offset e ignora (RNF05).
- **Ordem.** A chave por agregado preserva a ordem por pedido, reserva e evento. Consumidores toleram eventos tardios: por exemplo, o Checkout guarda `ReservaExpirada` mesmo que o pedido ainda não exista.
- **Mensagens venenosas.** Após 3 tentativas com backoff, a mensagem vai para o tópico `.dlq`, visível na Kafka UI, e o consumo segue.
- **Serviço fora do ar.** Se o emissao-service (emissão e notificações) cair, `PedidoPago` permanece retido no tópico e é processado na volta (RNF11); a venda não é afetada.

4.6 Fluxo ponta a ponta: da reserva ao ingresso

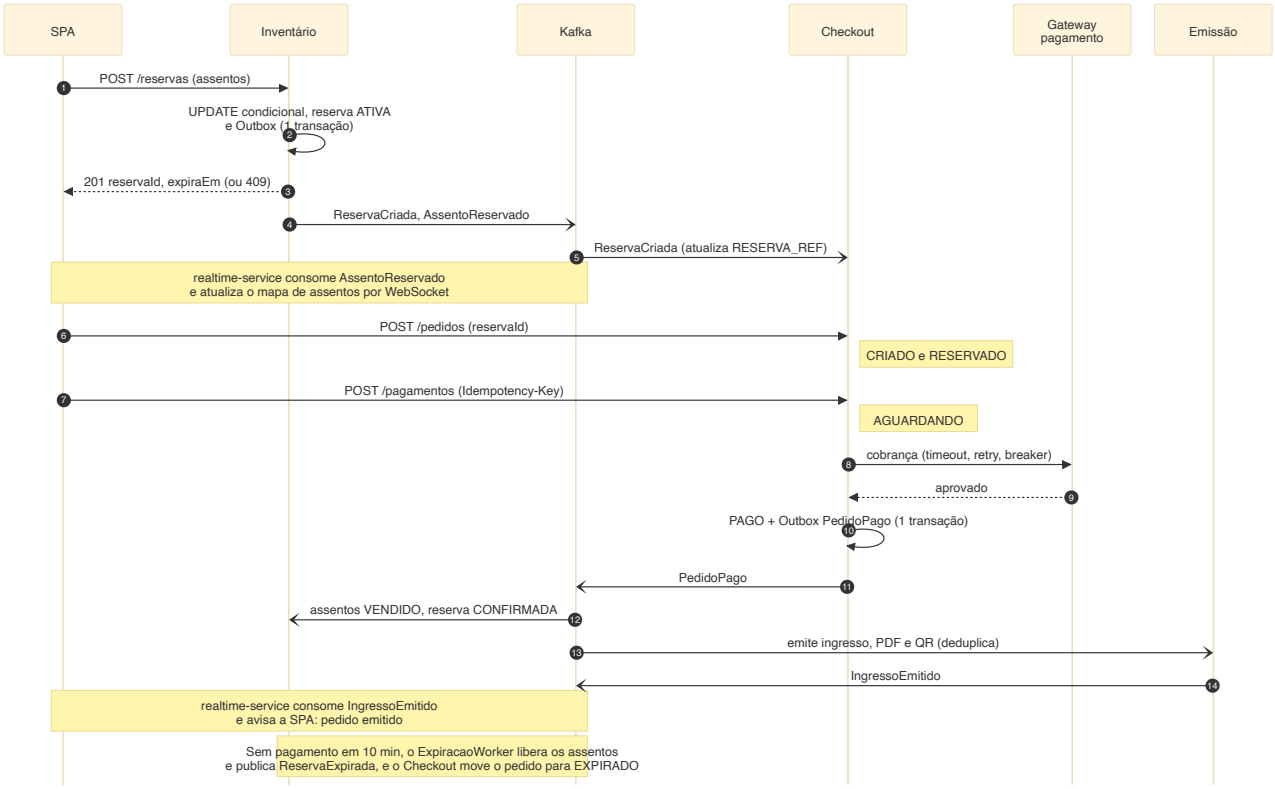


Figura 13 – Sequência do fluxo principal (R1) com publicação via Outbox.

5. Pipeline de Desenvolvimento

A pipeline é executada no GitHub Actions a cada Pull Request e a cada merge na branch `develop`. Os mesmos comandos podem ser executados localmente pelo Maven e pelo Docker Compose, de modo que o resultado do CI seja reprodutível na máquina de qualquer integrante (RNF19, RNF20).

5.1 Visão esquemática

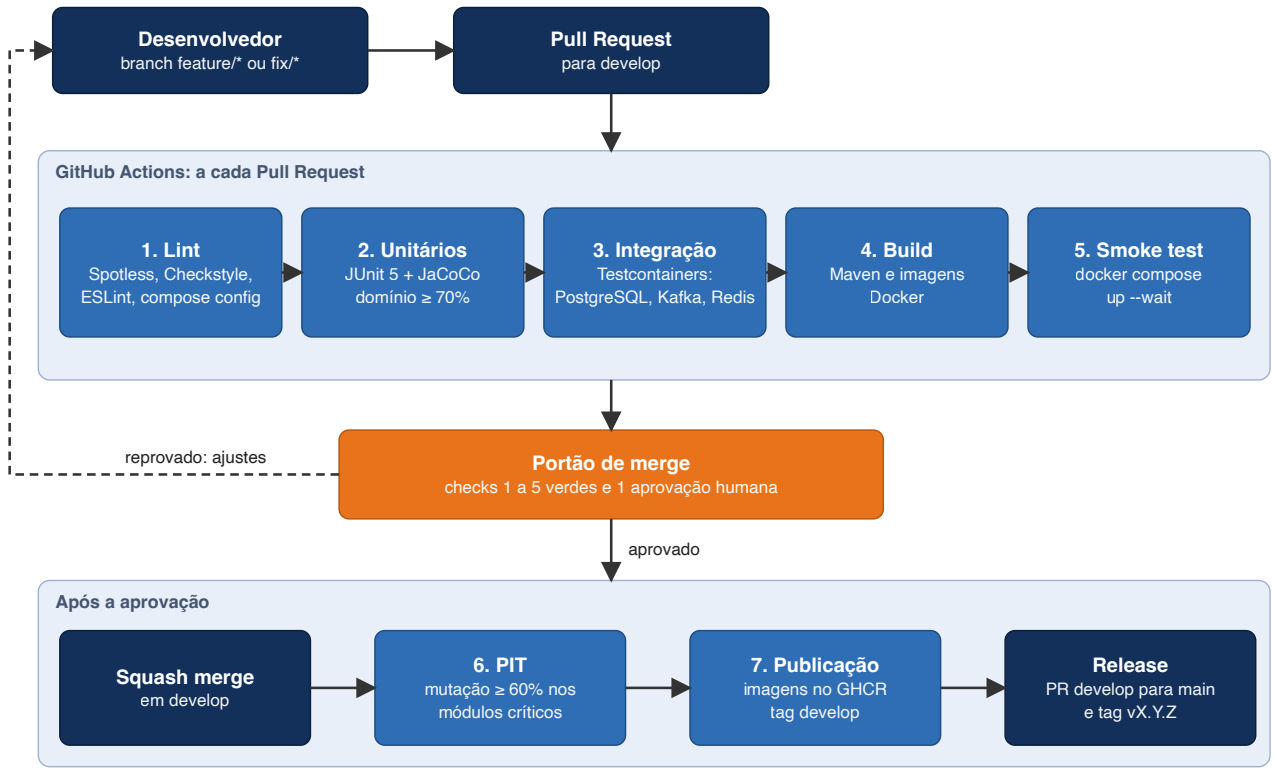


Figura 14 – Pipeline de CI/CD do PulsePass.

5.2 Etapas e portões de qualidade

#	Etapa	O que executa	Critério para passar
1	Lint	Spotless e Checkstyle no backend; ESLint e Prettier no frontend; validação do `docker-compose.yml` (`docker compose config`).	Sem violações; falha derruba a pipeline (RNF18).
2	Testes unitários	`mvn test` com JUnit 5, focado nas máquinas de estado de Reserva, Pedido, Pagamento e Ingresso; relatório JaCoCo publicado como artefato; Vitest no frontend.	Cobertura do domínio ≥ 70% (RNF17); testes escritos antes da implementação, quando gerados por agente.
3	Testes de integração	`mvn verify` com Testcontainers reais de PostgreSQL, Kafka e Redis: reserva concorrente (500 requisições, 1 sucesso), Outbox com Kafka indisponível, consumidor reprocessando o mesmo evento, pagamento com mesma `Idempotency-Key`.	Todos passam; RNF01, RNF03, RNF04 e RNF05 verificados.
4	Build	Compilação e empacotamento de cada serviço; build das imagens Docker, com cache de camadas e de dependências Maven.	Imagens construídas sem erro.

#	Etapa	O que executa	Critério para passar
5	Smoke test	`docker compose up --wait` (backend, frontend, bancos, Kafka, Kafka UI, Keycloak, observabilidade) e verificação dos *healthchecks* e de um fluxo mínimo de login e catálogo; auditoria Lighthouse/axe da SPA.	Todos os serviços saudáveis em até 3 min (RNF19); Lighthouse/axe ≥ 90 (RNF22).
6	Mutação (PIT)	Roda após o merge em `develop` e à noite, sobre os módulos críticos (domínio e Outbox), por ser mais lenta.	Mutação ≥ 60% (regra do Test Agent); resultado vira issue se cair abaixo.
7	Publicação	Publica imagens no GitHub Container Registry com tag `develop` (merge) ou `vX.Y.Z` (release).	Executa só após as etapas anteriores.

Verificações complementares: Dependabot para dependências, varredura de segredos (gitleaks) em cada PR e análise estática com SonarLint na IDE. **Testes de carga** (k6) com as metas dos RNF06 e RNF07 são um bônus: ficam em um workflow manual (`workflow_dispatch`) para não alongar o PR.

Proteção da branch. A branch `develop` exige o sucesso das etapas 1 a 5 como *required checks* (RNF20), o que bloqueia o merge de PR com pipeline vermelha.

5.3 Ambiente executável: Docker Compose

Serviço no Compose	Imagem ou origem	Observação
`frontend`	build local (Nginx)	Serve a SPA; porta 80.
`api-gateway`, `iam-service`, `inventario-service`, `checkout-service`, `emissao-service`, `telemetria-service`, `realtime-service`	build local (Maven)	`depends_on` com `condition: service_healthy`; réplicas configuráveis (RNF09).
`gateway-pagamento`	build local	Simulador; comportamento por variáveis de ambiente.
`postgres`	postgres:16	Um banco lógico por serviço, criados por script de inicialização.
`redis`	redis:7	Cache, locks e fila virtual.
`kafka` e `kafka-ui`	apache/kafka (KRaft) e UI de mensageria	Tópicos criados por um contêiner de inicialização, conforme a seção 4.2.
`keycloak`	Keycloak com *realm* importado	Papéis e usuários de teste no arquivo do realm.
`mailpit`	servidor SMTP de desenvolvimento	Captura os e-mails de ingresso.
`otel-collector`, `prometheus`, `grafana`	imagens oficiais	Traces, métricas e painéis (RNF16).

Variáveis sensíveis ficam em `.env` (ignorado pelo Git); o repositório traz apenas `.env.example` com valores de desenvolvimento.

6. Processo de Gerenciamento da Configuração de Software (GCS)

6.1 Ferramenta

Git para versionamento e GitHub como plataforma: repositório, Pull Requests, GitHub Projects (quadro Kanban e Story Map), Issues, Actions e Container Registry. Toda a configuração do software é versionada no repositório, sem artefatos fora dele.

6.2 Itens de configuração

Item	Local no repositório
Código-fonte dos serviços e da SPA	`/services/<servico>`, `/frontend`
Specs (requirements, design, tasks)	`/specs/<modulo>/<feature>`
Decisões arquiteturais (ADRs) e diagramas C4	`/docs/adr`, `/docs/arquitetura` (Mermaid)
Documentos dos pacotes de entrega	`/docs/pacotes`
Regras e prompts dos agentes de IA	`/.claude` ou `/.github`
Infraestrutura: Compose, Dockerfiles, *realm* do Keycloak, scripts de tópicos	`/infra`, `docker-compose.yml`
Pipelines	`/.github/workflows`

6.3 Fluxo de trabalho de branches

Branch	Uso	Regras
`main`	Versões estáveis, uma por pacote de entrega.	Só recebe PR vindo de `develop`; cada merge recebe uma tag.
`develop`	Integração contínua do trabalho.	Protegida: sem *push* direto, sem *force push*; PR com pipeline verde e 1 aprovação.
`feature/<id>-<descricao>`	Uma história de usuário, por exemplo `feature/US15-reserva-atomica`.	Criada a partir de `develop`; vida curta (poucos dias); removida após o merge.
`fix/<id>-<descricao>`	Correção de defeito.	Mesmas regras de `feature/*`.

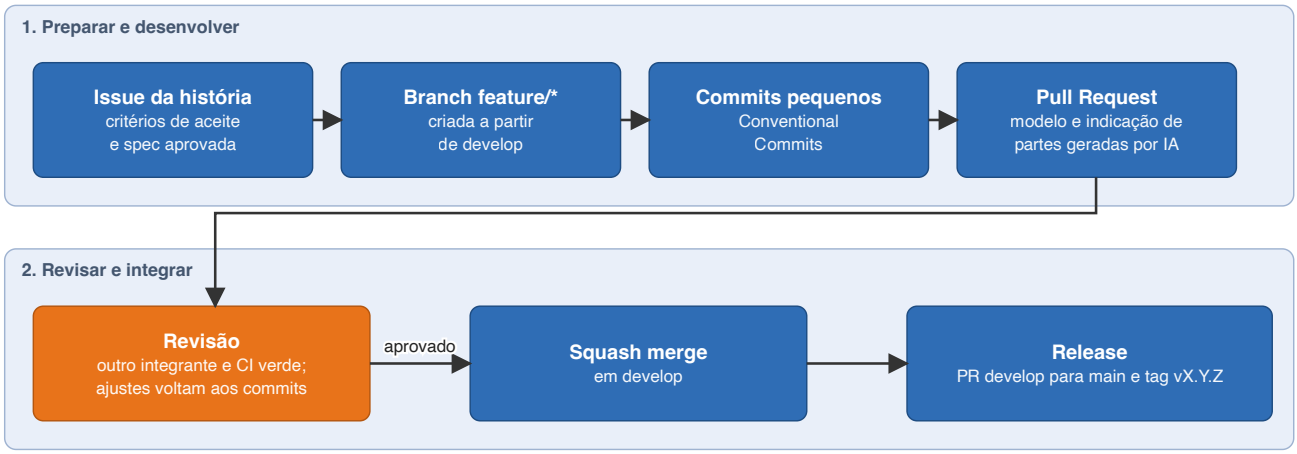


Figura 15 – Fluxo de uma mudança, da issue à release.

6.4 Regras de uso

1. **Todo trabalho nasce de uma issue** ligada ao épico (milestone) e à release (R1, R2, R3), com critérios de aceite e spec aprovada (Definition of Ready do Pacote 1).
2. **Commits** seguem Conventional Commits (`feat:`, `fix:`, `docs:`, `test:`, `chore:`) e citam a issue (`feat(inventario): reserva atômica (#15)`).
3. **Pull Request obrigatório** para `develop` e `main`. O modelo de PR traz: história, resumo, como testar, **quais partes foram geradas por IA** e a checklist da Definition of Done.
4. **Revisão:** o aprovador deve ser integrante diferente do autor. Agentes de IA comentam, mas nunca aprovam nem fazem merge.
5. **Merge por squash**, mantendo histórico linear em `develop`; o título do PR vira a mensagem do commit.
6. **Mudança de requisito ou de contrato:** altera primeiro a spec ou o ADR no mesmo PR; contratos de evento só mudam por adição de campos opcionais, ou por um novo tópico `v2`.
7. **Segredos** nunca são versionados: `.env` está no `.gitignore`; a pipeline usa GitHub Secrets; a varredura de segredos é um check obrigatório.
8. **Responsáveis por código:** arquivo `CODEOWNERS` indica o responsável por cada serviço; o PR solicita a revisão automaticamente.

6.5 Versionamento, linhas de base e controle de mudanças

- **Versionamento semântico** por tags em `main`: `v0.2.0` para o Pacote 2 (arquitetura e infraestrutura), `v0.3.0` para o checkpoint do Pacote 3 e `v1.0.0` para o produto completo do Pacote 4.
- **Linha de base:** cada tag congela o código, as specs, os ADRs e a documentação daquele pacote. Os pacotes 3 e 4 atualizam os documentos dos pacotes 1 e 2 por PR, mantendo o histórico das versões.
- **Controle de mudanças:** mudança proposta → issue com rótulo `mudanca` → impacto avaliado na spec e nos RNFs → aprovada por outro integrante → implementada por PR. Mudanças de arquitetura geram um novo ADR numerado.
- **Auditoria:** o histórico do Git, os PRs e a execução da pipeline são o registro de quem mudou o quê, quando, por qual motivo e com qual evidência de teste.

7. Rastreabilidade com o enunciado do Pacote 2

Entregável exigido	Onde está neste documento
Descrição da arquitetura no modelo C4 (Contexto, Contêineres, Componentes)	Seção 2: Figuras 1 (Nível 1), 2 (Nível 2) e 3 a 7 (Nível 3, um por serviço de negócio)
Descrição do Modelo de Dados Conceitual	Seção 3: Figuras 8 a 11 (visão integrada e por serviço), entidades de Telemetria e referências entre serviços (3.2)
Descrição da estrutura de mensageria do Kafka	Seção 4: tópicos (4.2), mapa (Figura 12), Outbox e idempotência (4.5), sequência (Figura 13)
Descrição esquemática da Pipeline de desenvolvimento	Seção 5: Figura 14, etapas e portões (5.2) e Docker Compose (5.3)
Descrição do Processo de GCS	Seção 6: ferramenta, itens de configuração, branches (Figura 15), regras e versionamento

Os requisitos do enunciado sobre *Dual-Write* e idempotência no consumo são tratados nas ADRs 003 e 004 e na seção 4.5.

8. Atualizações do Pacote 3

Este documento foi entregue em 04/11/2026 como desenho. O Pacote 3 implementou a fatia vertical R1 (catálogo, reserva atômica, pedido, pagamento idempotente, ingresso e validação), a SPA e a observabilidade, e o desenho foi seguido com as diferenças abaixo. As seções 1 a 7 permanecem como o desenho de referência; esta seção registra o que mudou ao construir.

Implementado e verificado: ADR-001 a ADR-004, ADR-007 e ADR-008; Outbox com `FOR UPDATE SKIP LOCKED` e consumo idempotente (`evento_processado`); máquinas de estado de reserva, assento, pedido, pagamento e ingresso; reserva com `UPDATE` condicional e filtro Redis; pagamento com `Idempotency-Key`, retry e circuit breaker; QR assinado; SPA; OpenTelemetry, Prometheus e Grafana; pipeline, Compose e processo de GCS descritos nas seções 5 e 6.

Ainda não implementado (Pacote 4): iam-service, telemetria-service, realtime-service, e-mail, fila virtual e rate limit no gateway. A tabela abaixo detalha as diferenças; o estado de cada história e de cada requisito não funcional, com a evidência de teste, está no documento do Pacote 3.

Seção do Pacote 2	Desenhado	Como ficou
2.3.2 Checkout	`IdempotencyFilter` como filtro servlet	A lógica está em `PagamentoService` (TX1, gateway fora de transação, TX2). Mesmas regras; chamada concorrente com a mesma chave enquanto a primeira roda recebe 409.
2.3.2 Checkout	Falha do gateway	Responde 503; o pagamento falho passa a ter chave própria (`#falha:id`) para a chave do cliente poder ser reutilizada. Recusa responde 402.
2.3.3 Emissão	`PdfStorage` e `NotificacaoWorker` / `EmailSender`	O PDF é gerado sob demanda, sem armazenamento (serviço continua sem estado). E-mail e a tabela `NOTIFICACAO` ficam para o Pacote 4 (US25 é R2).
2.2 Contêineres	SPA	Implementada em TypeScript (Vite) e servida por Nginx, que também faz proxy de `/api` para o gateway. Atualiza a disponibilidade por *polling* de 5 s, e não por WebSocket.
2.2 Contêineres	iam-service, telemetria-service e realtime-service	Não implementados. Os papéis vêm direto do Keycloak (`realm_access.roles`), e cada serviço valida o JWT pelo JWKS. O cadastro de clientes é o auto-cadastro do Keycloak, com o papel cliente por padrão.
2.2 Contêineres	API Gateway com rate limit	O gateway valida JWT e roteia sob o prefixo `/api`; responde 503 quando o serviço de destino está fora do ar. O rate limit não foi implementado.
2.3.1 Inventário	`FilaController` e fila virtual	Não implementados. O Redis é usado só para o cache do catálogo e para o `SeatLock`.
3 Dados	Tabela de Outbox	`outbox_event` guarda `topic`, `msg_key`, o envelope completo em `body` e o `traceparent`. A reserva ganhou `suspensa_ate` (ADR-008) e o lote tem `check` contra sobre-venda. O ingresso guarda `evento_nome` e `descricao` copiados do pedido.
4 Kafka	Criação de tópicos	Um contêiner `kafka-init` cria os tópicos com as partições da seção 4.2 e a criação automática está desligada. Sem isso, o consumidor que subia primeiro fazia o Kafka criar o tópico com 1 partição (defeito encontrado no smoke). Existem os tópicos de reserva, assento, pedido, ingresso e três DLQs; os de IAM e telemetria não foram criados.
4 Kafka	DLQ	Três tentativas com intervalo de 500 ms e depois o tópico `<tópico>.<grupo>.dlq`, na biblioteca `commons`.

Seção do Pacote 2	Desenhado	Como ficou
1.1 e 5.3 Observabilidade	OpenTelemetry, Prometheus e Grafana no Compose	Implementado: Micrometer Tracing com exportação OTLP para o Collector e o Jaeger; Prometheus com descoberta por DNS (encontra todas as réplicas); Grafana com dashboard provisionado; logs JSON. O `traceparent` é gravado na linha do Outbox e reaplicado pelo relay, para o trace atravessar a publicação assíncrona.
5 Pipeline	Etapas	Workflow `.github/workflows/ci.yml` com lint, testes, front-end, smoke, varredura de segredos, PIT (perfil Maven `mutacao`, no push em develop e à noite) e publicação no GHCR. O Compose não tem Mailpit.
1.1 Stack	Versões	Spring Boot 3.5.16, Spring Cloud 2025.0.3, Resilience4j 2.4.0 (uso programático), PDFBox 3.0.8, ZXing 3.5.4, Testcontainers 2.x, Keycloak 26.0, Vite 6, TypeScript 5.9, keycloak-js 26, Jaeger 1.62, Prometheus 2.55, Grafana 11.3.